Введение в списки и анализ данных

Реальные данные — это всегда наборы значений: список температур за месяц, результаты тестов, набор пользовательских оценок. Хранить их в отдельных переменных неудобно и ненадёжно.

Список (list) — главная структура данных для начала работы с данными в Python. Освоив его, вы сможете написать программу, которая принимает произвольный набор чисел и находит среднее, максимум, количество элементов, удовлетворяющих условию.

Из этого материала вы узнаете, как работать с индексацией и срезами, перебирать и фильтровать данные, а также решать задачи первичного анализа.

После этого занятия вы сможете:

- создавать списки чисел и строк, обращаться к элементам по индексу;
- применять срезы (slice) для получения части списка;
- перебирать список с индексом и без;
- фильтровать данные условием внутри цикла;
- вычислять `min`, `max`, среднее, количество элементов с условием;
- читать и трассировать программы со списками: определять без запуска, что выведет программа (такие задания встречаются в ЕГЭ по информатике).

Содержание

3.1. Создание и основные свойства
3.2. Индексация
3.3. Срезы (slice)
3.4. Основные методы и функции
3.5. Перебор: несколько способов
3.6. Фильтрация и накопление

3.7. Ввод списка с клавиатуры
Памятка: срезы

3.1. Создание и основные свойства

Создавать списки можно разными способами. Вот основные примеры:

```
# Список чисел
temps = [18, 22, 19, 25, 17, 30, 28]

# Список строк
subjects = ["Математика", "Физика", "Python", "История"]

# Смешанный список: Python его допускает, но sum(), min(), max()
# и sort() с ним не работают — числа и строки нельзя складывать и сравнивать
mixed = [42, "hello", True, 3.14]

# Пустой список — для последующего заполнения
data = []
```

3.2. Индексация

Индексы начинаются с `0`. Отрицательные индексы считают с конца.

Список `temps`	18	22	19	25	17	30	28
Прямой индекс	0	1	2	3	4	5	6
Обратный индекс	-7	-6	-5	-4	-3	-2	-1

```
temps = [18, 22, 19, 25, 17, 30, 28]
print(temps[0]) # 18 — первый
print(temps[-1]) # 28 — последний
print(temps[-2]) # 30 — предпоследний
```

3.3. Срезы (slice)

Срез позволяет получить часть списка: `list[start:stop:step]`.

- `start` — начальный индекс (включительно, по умолчанию `0`);
- `stop` — конечный индекс (не включается, по умолчанию до самого конца);
- `step` — шаг (по умолчанию `1`).

```
a = [10, 20, 30, 40, 50, 60, 70]
print(a[1:4]) # [20, 30, 40] – индексы 1, 2, 3
print(a[:3]) # [10, 20, 30] – первые три
print(a[4:]) # [50, 60, 70] – с 4-го до конца
print(a[::2]) # [10, 30, 50, 70] – каждый второй
print(a[::-1]) # [70, 60, 50, 40, 30, 20, 10] – задом наперёд
```

Важно помнить

Срез всегда возвращает **НОВЫЙ** список, не меняя исходный. Запись `a[1:4]` — это индексы 1, 2, 3 (индекс 4 не входит, точно так же, как в функции `range`).

3.4. Основные методы и функции

Операция	Синтаксис	Результат / действие
Длина	`len(lst)`	Количество элементов
Сумма	`sum(lst)`	Сумма всех элементов
Минимум	`min(lst)`	Наименьший элемент
Максимум	`max(lst)`	Наибольший элемент
Среднее	`sum(lst)/len(lst)`	Среднее арифметическое (только для непустого списка)
Добавить в конец	`lst.append(x)`	Добавляет x в конец

Удалить по значению	`lst.remove(x)`	Удаляет первое найденное x; если x в списке нет — ошибка `ValueError`
Сортировка	`lst.sort()`	Сортирует на месте (по возрастанию)
Отсортированная копия	`sorted(lst)`	Возвращает новый отсортированный список, исходный не меняется
Обратная сортировка	`lst.sort(reverse=True)`	Сортирует на месте (по убыванию)
Индекс элемента	`lst.index(x)`	Позиция первого найденного x; если x в списке нет — ошибка `ValueError`
Число вхождений	`lst.count(x)`	Сколько раз x встречается в списке
Вхождение	`x in lst`	Возвращает `True` или `False`

Ограничения функций

`sum()` работает только со списком чисел. `min()`, `max()` и `sort()` требуют, чтобы элементы можно было сравнить между собой: либо все числа, либо все строки. Для пустого списка `min()` и `max()` вызывают ошибку `ValueError`, а `sum(lst)/len(lst)` — ошибку деления на ноль `ZeroDivisionError`. Поэтому перед вычислением среднего проверяйте, что список не пуст: `if len(lst) > 0:`. Перед `remove(x)` и `index(x)` удобно проверить `if x in lst:`.

sort() или sorted()

`lst.sort()` меняет сам список и ничего не возвращает (результат вызова — `None`), поэтому запись `b = lst.sort()` — ошибка. `sorted(lst)` создаёт новый отсортированный список, а исходный оставляет без изменений: `b = sorted(lst)`.

```
nums = [3, 1, 2]
b = sorted(nums)
print(b)     # [1, 2, 3] – новый список
print(nums)  # [3, 1, 2] – исходный не изменился
nums.sort()
print(nums)  # [1, 2, 3] – изменился сам список
```

3.5. Перебор: несколько способов

Перебирать элементы списка можно разными способами в зависимости от задачи:

```
scores = [85, 92, 71, 88, 60]

# Способ 1 – по значению (проще, если индекс не нужен)
for s in scores:
    print(s)

# Способ 2 – по индексу (когда нужен номер позиции)
for i in range(len(scores)):
    print(i, scores[i])

# Способ 3 – enumerate (индекс + значение одновременно)
for i, s in enumerate(scores):
    print(f"Позиция {i}: {s}")
```

3.6. Фильтрация и накопление

С помощью условий внутри цикла можно отбирать нужные элементы и собирать их в новые списки или считать их количество.

```
scores = [85, 92, 71, 88, 60, 95, 58]

# Подсчёт элементов, удовлетворяющих условию
count_pass = 0
for s in scores:
    if s >= 70:
        count_pass += 1
print("Сдали:", count_pass)

# Сбор подходящих элементов в новый список
passed = []
for s in scores:
    if s >= 70:
        passed.append(s)
print("Результаты сдавших:", passed)
```

Пример задачи

Условие. Дан список оценок — целые числа от 0 до 100. Выведите количество оценок не ниже 70.

Входные данные: `scores = [85, 92, 71, 88, 60, 95, 58]`.
Выходные данные: одна строка вида `Сдали: 5`.
Если список пуст, программа выводит `Сдали: 0` — цикл не выполнится ни разу.

3.7. Ввод списка с клавиатуры

Чаще всего данные не записаны в программе заранее, а вводятся пользователем. Есть два основных способа.

```
# Способ 1 – сначала количество, затем каждое число в отдельной строке
n = int(input())
nums = []
for i in range(n):
    nums.append(int(input()))
print(nums)

# Способ 2 – все числа в одной строке через пробел, например: 5 3 8
line = input()
nums = []
for part in line.split():
    nums.append(int(part))
print(nums)
```

Как это работает

Метод `split()` разрезает строку по пробелам и возвращает список строк: `"5 3 8".split()` → `["5", "3", "8"]`. Каждую часть нужно превратить в число с помощью `int()`. Если ввести не число (например, букву), `int()` вызовет ошибку `ValueError`.

Памятка: срезы

Запись	Смысл	Пример (для `a = [0, 10, 20, 30, 40, 50, 60]`)
`a[i]`	Элемент с индексом `i`	`a[2]` → `20`
`a[i:j]`	Элементы от `i` до `j` (не включая `j`)	`a[1:4]` → `[10, 20, 30]`
`a[:j]`	От начала до `j`	`a[:3]` → `[0, 10, 20]`
`a[i:]`	От `i` до конца	`a[4:]` → `[40, 50, 60]`

`a[::k]`	Каждый k-й элемент	`a[::2]` → `[0, 20, 40, 60]`
`a[::-1]`	Список в обратном порядке	`a[::-1]` → `[60, 50, 40, 30, 20, 10, 0]`
`a[-1]`	Последний элемент	`a[-1]` → `60`
`a[-3:]`	Последние три элемента	`a[-3:]` → `[40, 50, 60]`

Главное правило: списки — это мощный инструмент для работы с наборами данных, а срезы позволяют легко получать нужные части списка без изменения оригинала.